

Doctor Appointment Management System

Project Overview

This is a new full-stack project.

The goal is to build a simple **Patient, Doctor, and Appointment Management System** using **TypeScript**.

Your previous project focused on learning the basics of:

- React
- Node.js
- Express
- CRUD operations
- Frontend and backend communication

This project is the next step.

In addition to building new features, the main focus should be on:

- Learning TypeScript
- Organizing frontend code properly
- Keeping backend code simple and organized
- Working with related data
- Understanding relationships between Patients, Doctors, and Appointments
- Using environment variables correctly
- Keeping sensitive information out of GitHub

Please keep the application simple, clean, and understandable.

Technology Stack

Use the following technologies:

- TypeScript
- React
- React Router
- Node.js
- Express.js

- MongoDB or another database

Use TypeScript for both the frontend and backend.

Use:

- `.tsx` for React components and pages
- `.ts` for other TypeScript files

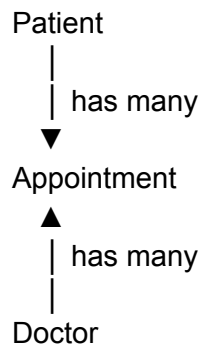
Try to avoid using `any` unless there is a valid reason.

Application Modules

The application will have three main modules:

1. Patient Management
2. Doctor Management
3. Appointment Management

The main relationship is:



Each appointment belongs to:

- One Patient
- One Doctor

A Patient can have multiple appointments.

A Doctor can also have multiple appointments.

1. Patient Management

Create a **Patients** page.

Users should be able to:

- Register a new patient
- View all registered patients
- Edit patient information
- Delete a patient
- View patient details

Each patient should have at least:

- Name
- Phone
- Age
- Gender

Example TypeScript interface:

```
interface Patient {  
  id: string;  
  name: string;  
  phone: string;  
  age: number;  
  gender: string;  
}
```

Patient List

The Patients page should display a list of all registered patients.

For each patient, provide the following actions:

- Edit
- Delete
- Details

Example:

Name	Phone	Age	Gender	Actions
------	-------	-----	--------	---------

Rahim	017XXXXXXXX	35	Male	Edit Delete Details
Karim	018XXXXXXXX	42	Male	Edit Delete Details

Clicking the **Details** button should navigate to the Patient Details page.

2. Patient Details Page

Every patient should have a Details page.

The Patient Details page should display the complete information of the selected patient.

For example:

Patient Details

Name: Rahim

Phone: 017XXXXXXXX

Age: 35

Gender: Male

The page should also display **all appointments for that patient**.

For each appointment, display:

- Doctor Name
- Doctor Specialization
- Patient Problem / Reason for Visit
- Appointment Date

Example:

Appointments

Doctor: Dr. John Smith

Specialization: Cardiology

Problem: Chest Pain

Appointment Date: 20 August 2026

Doctor: Dr. Sarah Ahmed

Specialization: Medicine

Problem: High Blood Pressure
Appointment Date: 25 August 2026

The appointment data should come from the backend.

3. Doctor Management

Create a **Doctors** page.

Users should be able to:

- Register a new doctor
- View all registered doctors
- View doctor details

Each doctor should have at least:

- Name
- Phone
- Age
- Gender
- Specialization

Example TypeScript interface:

```
interface Doctor {  
  id: string;  
  name: string;  
  phone: string;  
  age: number;  
  gender: string;  
  specialization: string;  
}
```

Doctor List

The Doctors page should display all registered doctors.

For each doctor, provide a **Details** button.

Example:

Name	Specialization	Actions
Dr. John Smith	Cardiology	Details
Dr. Sarah Ahmed	Medicine	Details

Clicking the **Details** button should navigate to the Doctor Details page.

4. Doctor Details Page

The Doctor Details page should display the complete information of the selected doctor.

For example:

Doctor Details

Name: Dr. John Smith
Phone: 017XXXXXXXX
Age: 45
Gender: Male
Specialization: Cardiology

The page should also display **all appointments for that doctor**.

For each appointment, display:

- Patient Name
- Patient Age
- Patient Gender
- Patient Problem
- Appointment Date

Example:

Appointments

Patient: Rahim
Age: 35
Gender: Male
Problem: Chest Pain
Appointment Date: 20 August 2026

Patient: Karim
Age: 42
Gender: Male
Problem: High Blood Pressure
Appointment Date: 22 August 2026

The appointment data should come from the backend.

5. Appointment Management

Create an **Appointments** page.

An appointment can only be created if both the Patient and Doctor are already registered.

While creating an appointment, the user should be able to select:

- Registered Patient
- Registered Doctor
- Problem / Reason for Visit
- Appointment Date

The Patient and Doctor should be selected from the existing registered lists.

Example:

Create Appointment

Patient:
[Select Patient ▼]

Doctor:
[Select Doctor ▼]

Problem:
[_____]

Appointment Date:
[_____]

[Create Appointment]

Do not manually type the Patient or Doctor name while creating an appointment.

The appointment should store the relationship between the selected Patient and Doctor.

Example:

```
interface Appointment {  
    id: string;  
    patientId: string;  
    doctorId: string;  
    problem: string;  
    appointmentDate: string;  
}
```

6. Appointment List

The application should have a page where all appointments can be viewed.

Display at least:

- Patient Name
- Doctor Name
- Problem
- Appointment Date

Example:

Patient	Doctor	Problem	Appointment Date
Rahim	Dr. John Smith	Chest Pain	20 August 2026
Karim	Dr. Sarah Ahmed	Headache	21 August 2026

The appointment data should come from the backend API.

Application Pages

Your application should have the following pages:

Patients

- Patient List
- Add Patient
- Edit Patient
- Patient Details

Doctors

- Doctor List
- Add Doctor
- Edit Doctor
- Doctor Details

Appointments

- Create Appointment
- Appointment List

You should use React Router for navigation.

A simple navigation can be:

[Patients] [Doctors] [Appointments]

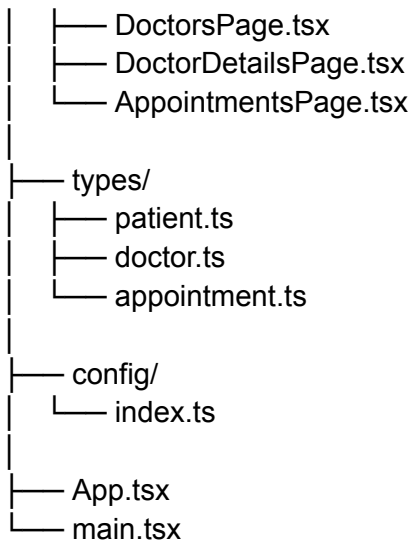
Frontend Folder Structure

Do not put all the application code inside `App.tsx`.

Use a simple and organized folder structure.

Suggested structure:

```
client/
├── src/
│   ├── components/
│   │   ├── PatientForm.tsx
│   │   ├── PatientList.tsx
│   │   ├── DoctorForm.tsx
│   │   ├── DoctorList.tsx
│   │   ├── AppointmentForm.tsx
│   │   └── AppointmentList.tsx
│   └── pages/
│       ├── PatientsPage.tsx
│       └── PatientDetailsPage.tsx
```



Frontend Folder Responsibilities

components

Contains reusable UI components.

For example:

- Patient form
- Patient list
- Doctor form
- Doctor list
- Appointment form
- Appointment list

pages

Contains the main pages of the application.

For example:

- Patients page
- Patient Details page
- Doctors page
- Doctor Details page
- Appointments page

services

Contains API calls.

For example:

patientService.ts

Should contain Patient-related API calls.

doctorService.ts

Should contain Doctor-related API calls.

appointmentService.ts

Should contain Appointment-related API calls.

types

Contains TypeScript types and interfaces.

For example:

- Patient
- Doctor
- Appointment

config

Contains application configuration.

For example, the API base URL.

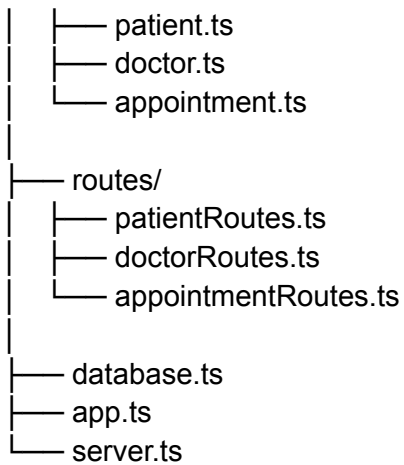
Backend Folder Structure

Keep the backend structure very simple.

Do not create unnecessary folders.

Use the following structure:

```
server/  
├── src/  
│   └── models/
```



For this project, keep the API logic inside the relevant route files.

The goal is to learn basic project organization without making the backend unnecessarily complex.

Backend Responsibilities

models

Contains the database models.

You should have:

- Patient model
- Doctor model
- Appointment model

The Appointment model should store references to:

- Patient
 - Doctor
-

routes

Contains API endpoints and request handling logic.

For this project, you can keep the route logic inside the route files.

For example:

patientRoutes.ts

Contains Patient-related APIs.

doctorRoutes.ts

Contains Doctor-related APIs.

appointmentRoutes.ts

Contains Appointment-related APIs.

app.ts

Configure the Express application.

For example:

- `express.json()`
 - CORS
 - Registering routes
-

server.ts

Start the Express server.

This file should mainly be responsible for starting the application.

Expected API Endpoints

Patient APIs

GET /api/patient-list/get

POST /api/patient/create

GET /api/patient/:id/get
PUT /api/patient/:id/update
DELETE /api/patient/:id/delete

To get all appointments for a specific patient:

GET /api/patient/:id/appointment-list/get

Doctor APIs

GET /api/doctor-list/get
POST /api/doctor/create
PUT /api/doctor/:id/update
GET /api/doctor/:id/get

To get all appointments for a specific doctor:

GET /api/doctor/:id/appointment-list/get

You may add Doctor update and delete APIs as an optional improvement.

Appointment APIs

GET /api/appointment-list/get
POST /api/appointment/create

The **POST** API should accept:

patientId
doctorId
problem
appointmentDate

Before creating an appointment, make sure:

- The selected Patient exists.
 - The selected Doctor exists.
-

Environment Variables

Do not hardcode environment-specific values.

For example, avoid this:

```
const API_BASE_URL = "http://localhost:3000/api";
```

Instead, use environment variables.

Frontend

Example `.env`:

```
VITE_API_BASE_URL=http://localhost:3000/api
```

Use it like this:

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL;
```

Backend

Example `.env`:

```
PORT=3000  
DATABASE_URL=your_database_connection_string
```

Important: Do Not Push `.env` to GitHub

Never push the actual `.env` file to GitHub.

The `.env` file may contain sensitive information such as:

- Database connection strings
- Passwords
- API keys
- Secret keys

Add `.env` to `.gitignore`:

```
.env
```

Also create a `.env.example` file.

Example:

PORT=
DATABASE_URL=
VITE_API_BASE_URL=

The `.env.example` file can be pushed to GitHub because it only contains the names of the required environment variables, not real credentials.

TypeScript Requirements

This project must use TypeScript.

The goal is not simply to rename `.js` files to `.ts`.

Use TypeScript to define and understand the structure of your data.

Create proper types or interfaces for:

- Patient
- Doctor
- Appointment

You should also try to use TypeScript for:

- Component props
- Function parameters
- Function return values
- API response data
- State data where appropriate

Avoid using:

```
const data: any = {};
```

unless there is a valid reason.

The purpose of this project is to start understanding:

- Interfaces
- Types
- Function parameter types
- Function return types
- Typed React props
- Typed API data

